

CSCI212: Lecture #7 (Professor Ryba's lecture)

Kevin Bedoya

Wednesday 2/18/26

Review

- How Java stores objects
- References, aliases, garbage
- How do objects behave as parameters?
- Standard methods for all objects (Java will make them)
- How arrays behave like objects

The *new* keyword in Java

- Is used to make new objects
- Runs the constructor for the object
- Finds space in the **heap**, places the object there and gives us a reference to find it.

In Java memory (ways to interpret how objects are stored in memory/ *reference semantics*)

Stack	Heap
myAccount	[Alex, 20].

Example usage of aliases

```
criminalAccount = myAccount
```

criminalAccount is now an **alias** for the object in *myAccount*. In general, the "=" operator in Java **ALWAYS** sets up an alias!

Standard methods for all objects

Two standard methods for objects:

- equals : tells us when objects are equal.
- toString : makes a String for printing with *object info*.

Implementing *toString()* for an object

In this example, the *toString()* method is implemented for the *BankAccount* class discussed in **lecture #6**.

```
1 public String toString(){
2     return this.name + " " + this.balance;
3 }
```

After implementing the *toString()* method, the object will no longer be referenced by its exact memory location, when using:

```
1 BankAccount myAccount, criminalAccount;
2 myAccount = new BankAccount("Alex", 20.0); // object creation
3
4 System.out.println("\n" + myAccount);
```

Instead, the object will be referenced by the contents printed by *toString()*.

Alex 20.0

We will implement a static method titled, *chargeFee()*

```
1 private static void chargeFee(BankAccount criminalAccount, double fee) {
2     criminalAccount.balance -= fee;
3 }
```

All content in the *main()* method

```
1 public static void main(String[] args){
2     BankAccount myAccoun, criminalAccount;
3     myAccount = new BankAccount("Alex", 20.0);
4     System.out.println("\n" + myAccount);
5     criminalAccount = new BankAccount("Bad Guy", 20.0);
6     print2Acct(myAccount, criminalAccount);
7     myAccount.deposit(100);
8     criminalAccount.withdraw(1000.0);
9     print2Acct(myAccount, criminalAccount);
10    criminalAccount = myAccount;
11    print2Acct(myAccount, criminalAccount);
12    criminalAccount.setName("BadGuy");
13    System.out.println(criminalAccount);
14    chargeFee(criminalAccount, 50);
15    System.out.println(criminalAccount);
16 }
```

Passing objects as parameters to functions

When an object is passed as a parameter to a function, the function can change its data. Objects in Java behave as if passed by reference to functions. Java *copies* the references.

Facts on arrays in Java

In Java you can make an array out of primitive data or objects.

```
1 int a[]; // A is an array of integers (primitive)
2 BankAccount bank[]; //bank is an array of bank accounts (object)
```

References are made in the following way (on the heap):

```
1 a = new int[5];
2 // Find space on the heap for 5 integers
```

```
1 bank = new BankAccount[4];  
2 // In order to create bank accounts, you need to use the "new" keyword  
3 bank[0] = new BankAccount("Alex", 20);  
4 // The constructor is called with the provided data  
5 bank[1] = new BankAccount("Marty", 1000);  
6 // You can also store existing bankAccounts (no need to instantiate another object  
  at a new index of the array)  
7 bank[2] = bank[1];
```

Arrays behave like objects.

End of lecture #7.